

tradeforme — Next Steps

Action list of things only **you** can do. Everything below is blocked on your input, your credentials, or your decision — not on more code.

Where the project stands

Repository	https://github.com/EtixP/tradeforme (private)
Branch	main
Milestones done	M1 (skeleton), M2 (DART), M3 (deterministic extraction, 96% success rate), M4 (event s
Tests passing	84 / 84
Disclosures in DB	111,622 across 60 trading days (Feb 27 – May 27, 2026)
Supply-contract events	533 candidates → 514 parsed (96.4% OK, 18 blocked, 1 review)
Real contract values extracted	514 events with verified contract_value, prior_year_revenue, ratio
Price data source	pykrx (free, no auth)
LLM extraction	Code complete (prompts + validator + client) — optional now that deterministic parser wo
Broker integration	PaperBroker (historical fills) ready; KIS broker needs your credentials

Threshold sweep: net-of-cost returns by ratio cutoff

514 contracts with real extracted values, returns measured from event-day close to T+1/T+5 close, net of 0.313% roundtrip cost. The CLAUDE.md hypothesis (filter to ratio ≥ 0.08) is the line marked « in the table.

Threshold	n	T+1 net	T+5 net	T+1 win%	T+5 win%	Profit factor (5d)
0.00 (all)	514	+0.35%	+1.21%	42.6%	44.2%	1.38
0.05	430	+0.43%	+0.95%	43.7%	42.8%	1.30
0.08 «	333	+0.39%	+0.55%	41.7%	42.3%	1.20
0.10	287	+0.54%	+0.87%	40.8%	44.3%	1.27
0.15	169	+0.18%	+0.89%	38.5%	46.2%	1.26
0.20	110	+0.49%	+1.15%	38.2%	45.5%	1.33
0.30	59	+1.64%	+3.10%	40.7%	42.4%	1.93
0.50	23	+1.78%	+2.67%	30.4%	39.1%	1.77

Key finding: the CLAUDE.md-suggested 8% threshold underperforms the unfiltered baseline (T+5 net +0.55% vs +1.21%). The 30%+ threshold shows real edge — T+5 net +3.10%, profit factor 1.93 — but on only 59 events, well within noise.

What this means: the project's actual research question now has a partial answer: small-to-medium supply contracts do NOT show a tradable post-disclosure drift, but very large ones ($\geq 30\%$ of revenue) might. To confirm, the system needs more data — *backfill* is now the highest-leverage next move.

Raw CSVs: `data/event_study_results.csv` (all 533) and `data/event_study_filtered.csv` (ratio ≥ 0.08).

Action items

Sorted by priority. P1 unblocks the next milestone. P2 hardens the existing system. P3 is optional polish.

Priority	What	Why	How
P1	Authorize a larger backfill (12+ months)	Strongest finding so far: very large crypto backfill 12-300k transactions for +6.16 months, not longer only 350k	Deploy to backfill 12-300k transactions for +6.16 months, not longer only 350k
P1	Rotate the DART API key	The current key briefly appeared in data maps during the ██████████ smoke test (██████████) Replace with ██████████	██████████
P2	Get an LLM API key (Anthropic)	Democratize AI on P1: optional stic parsers (see last 96%) of supply chain trace with LLM (add the)	██████████
P2	Set up GitHub branch protection	Once live trading code exists, you don't want to break it by accident	gitHub.com/StrayPylab/forman-to-Settable-LV-Branches by accident
P2	Back up data/kdtb.db	111k rows of ingested DART data	Easiest: rent a 100GB S3 bucket, backup 5 min the Desktop 90 days
P2	Schedule daily DART ingestion	Right now ingestion only runs when you invoke the CLI manually	Use cron or systemd timer to run CLI daily
P2	Obtain KIS broker credentials	Required for Milestone 6 (broker integration)	Go to KIS website, create account, get credentials
P2	Decide whether to raise min_confidence threshold	Data from this loop's threshold sweep	Open config file, find min_confidence, adjust if needed
P2	Review risk thresholds in config	Default: 0.7 with CLAUDE.md (██████████) 30k	Open config file, find risk thresholds, adjust if needed
P3	Install gitleaks pre-commit hook	Catches an accidental commit of .env	brew install gitleaks & gitleaks pre-commit
P3	Decide if PDF action lists should be generated	Currently METEOR_STEPS.pdf will be generated as a side effect of METEOR workflow	Decide if PDF action lists should be generated
P3	Decide on Korean market holidays	Big handoff: the ingestion CLI runs even on holidays	Optional: add a holiday calendar (public for holidays, private for business days)

Where to find things

Project root	/Users/jsp2022310/Desktop/tradeforme
Repo on GitHub	https://github.com/EtixP/tradeforme
Project spec	CLAUDE.md (in repo root)
Source code	src/kdtb/ — schemas, config, data, strategy, risk, storage
Tests	tests/ — run with: <code>.venv/bin/pytest</code>
DART ingestion CLI	<code>scripts/ingest_disclosures.py --date YYYY-MM-DD</code>
Event study CLI	<code>scripts/run_event_study.py [--limit N]</code>
Parse docs CLI (NEW)	<code>scripts/parse_supply_contracts.py [--limit N]</code>
Filtered event study (NEW)	<code>scripts/run_filtered_event_study.py [--min-ratio 0.08]</code>
Local DB (gitignored)	data/kdtb.db — SQLite, inspect with sqlite3 or DB Browser
Event study CSV	data/event_study_results.csv
Secrets (gitignored)	.env — DART_API_KEY lives here
Config	config/default.yaml, config/paper.yaml, config/live_tiny.yaml
Live-trading guard	src/kdtb/config.py — raises if YAML says LIVE without <code>ENABLE_LIVE_TRADING=true</code> in

How to verify the project is healthy

Run these from the repo root any time you want to confirm nothing is broken:

```
.venv/bin/pytest
```

```
.venv/bin/python -m kdtb.main
```

```
.venv/bin/python scripts/ingest_disclosures.py --date $(date +%Y-%m-%d)
```

What Claude built across the autonomous loops

Loop 1 (M4 + M5):

- **data/market_data_client.py** — pykrx wrapper for OHLCV + event-time returns
- **scripts/run_event_study.py** — batched event study CLI
- **strategy/major_supply_contract.py** — M5 strategy engine (was placeholder)

Loop 2 (cost model + M3 prep + paper broker):

- **backtest/cost_model.py** — Korean equity costs
- **backtest/metrics.py** — per-trade metrics
- **interpretation/llm_prompts.py, extraction_validator.py, llm_client.py**
- **broker/base.py + broker/paper_broker.py**

Loop 3 (deterministic extraction + filtered event study):

- **data/dart_client.py** — new fetch_document_text() method + ZIP/HTML/UTF-8 extractor
- **interpretation/deterministic_parser.py** — regex parser for the ■■■■■■■■■■■■ form (both voluntary and mandatory variants)
- **storage/extraction_store.py** — SQLite upsert keyed on (disclosure_id, model_name)
- **scripts/parsing_supply_contracts.py** — CLI: fetches docs → parses → stores. Ran clean on 533 events.
- **scripts/run_filtered_event_study.py** — merges prices with extractions, reports gross/net by ratio cutoff
- 3 new test files (16 new test cases)

Suite: 28 $\rightarrow$ 43 $\rightarrow$ 68 $\rightarrow$ **84** passing tests. Zero regressions across three loops.

Honest caveats

- The naive event study uses title-only filtering. It does not separate small contracts from large ones, so noise dominates. M3 (LLM extraction) is needed to filter by contract/revenue ratio.
- The event date used is the DART receipt date (YYYYMMDD). Intraday timing of the disclosure is not captured. If a disclosure is filed at 14:00 KST, our 'T+1' return actually includes the price reaction from 14:00 to next-day close, not a clean next-day-only return.
- Returns are raw close-to-close, not benchmark-adjusted. A market-wide rally on event days would inflate the mean.
- pykrx scrapes the KRX website. It's free but can be rate-limited. We sleep 0.15s between calls.
- LIVE_TINY broker code is not yet implemented. Even with KIS credentials, no real orders can be placed yet.